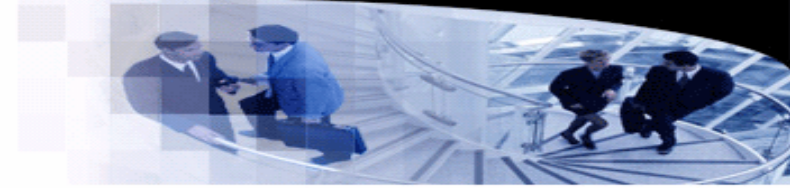




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

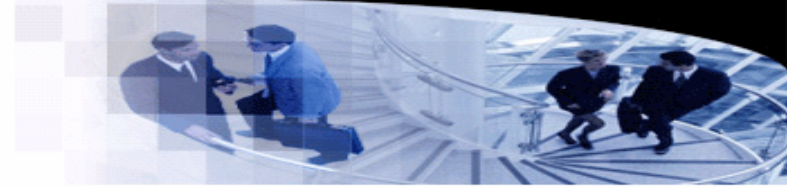
Lecture 7. Designing A Language

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

How to Design A Language



- **Syntax**

- Specify which strings from the alphabet are well-formed.

- **Semantics**

- Define the “meanings” of well-formed strings.

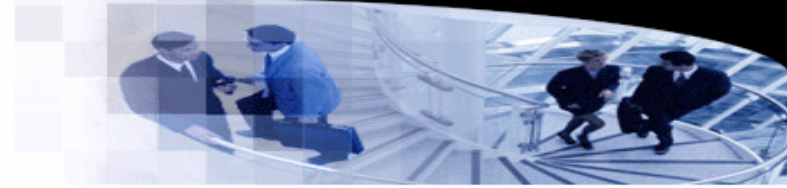
- **Types**

- Specifications of programs, in terms of types.

- **Toy language**

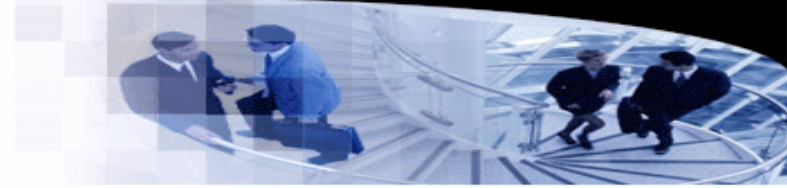
- In this lecture, a simple language will be presented to demonstrate core concepts of designing programming languages.

Toy Language



- A language of untyped arithmetic expressions
 - `true` and `false` are Boolean values.
 - `0` is the only one integer value.
 - `if t1 then t2 else t3` returns `t2` or `t3` based on the condition expression `t3`.
 - `succ t` and `pred t` are expressions that calculate the successor and the predecessor of `t`.
 - `iszero t` checks whether `t` is `0`.
- Syntax in BNF
 - `t ::= true | false | if t then t else t | 0 |`
 - `succ t | pred t | iszero t`
 - The language is the set of all well-formed terms (expressions) produced by `t`.
- Example
 - `if iszero succ 0 then true else 0`

Semantics



- **Evaluation**

- Given an arithmetic expression, what is the “meaning” of the expression?
- If we write an expression to calculate the result, the evaluation process can be considered as the meaning (or semantics, in another word) of the expression.
- The semantics in this lecture is given in the style of **operational semantics**.

- **Inference rules (rules of inference)**

- In this lecture, we leverage inference rules to define such evaluation / calculation / computation processes rigorously.
- An inference rule $\frac{\text{premise}(s)}{\text{conclusion}(s)}$ is a logical form that takes premises and draw a conclusion (or conclusions).

- **Example**

- $$\frac{A \rightarrow B \quad A}{B}$$

Semantics

- Boolean expressions

$\mathbb{B}$ (untyped)

Syntax

$t ::=$

`true`

`false`

`if t then t else t`

$v ::=$

`true`

`false`

terms:

constant true

constant false

conditional

values:

true value

false value

Evaluation

$t \rightarrow t'$

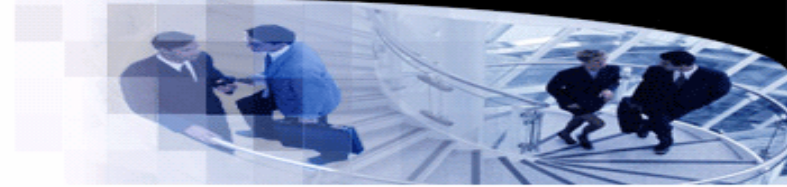
`if true then  $t_2$  else  $t_3 \rightarrow t_2$` (E-IFTRUE)

`if false then  $t_2$  else  $t_3 \rightarrow t_3$` (E-IFFALSE)

$$\frac{t_1 \rightarrow t'_1}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightarrow \text{if } t'_1 \text{ then } t_2 \text{ else } t_3}$$
 (E-IF)

- The left-hand side of the figure defines the syntax, and the right-hand side defines the semantics.
- $t \rightarrow t'$ denotes that t evaluates to t' in one step.

Semantics



- **Evaluation**

- An **instance** of an inference rule is obtained by consistently replacing each metavariable by the same term in the rule's conclusion and all its premises.
- Example:

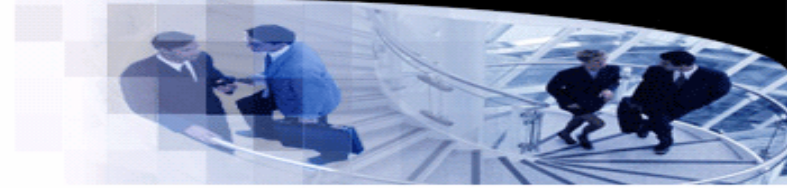
$\text{if true then } t_2 \text{ else } t_3 \rightarrow t_2$ (E-IFTRUE)

- Instance: $\text{if true then true else false} \rightarrow \text{true}$

$$\frac{t_1 \rightarrow t'_1}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightarrow \text{if } t'_1 \text{ then } t_2 \text{ else } t_3}$$
 (E-IF)

- Instance:
$$\frac{\text{true} \rightarrow \text{false}}{\text{if true then true else false} \rightarrow \text{if false then true else false}}$$

Semantics



- **Evaluation**

- A rule is **satisfied** by a relation if, for each instance of the rule, either the conclusion is in the relation or one of the premises is not.
- The **one-step evaluation relation** $\rightarrow$ is the smallest binary relation on terms satisfying the three evaluation rules.
- When the pair (t, t') is in the evaluation relation, we say that “the evaluation statement (or judgment) $t \rightarrow t'$ is derivable.”
- Example:

$s \stackrel{\text{def}}{=} \text{if true then false else false}$

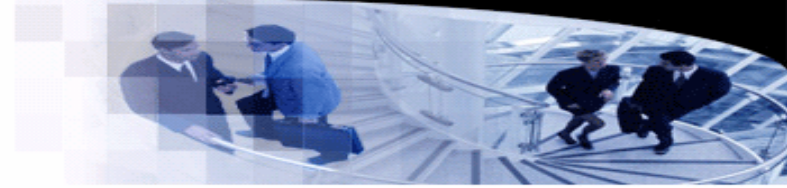
$t \stackrel{\text{def}}{=} \text{if } s \text{ then true else true}$

$u \stackrel{\text{def}}{=} \text{if false then true else true}$

$$\frac{\frac{}{s \rightarrow \text{false}} \text{E-IFTRUE}}{t \rightarrow u} \text{E-IF}$$

$$\frac{}{\text{if } t \text{ then false else false} \rightarrow \text{if } u \text{ then false else false}} \text{E-IF}$$

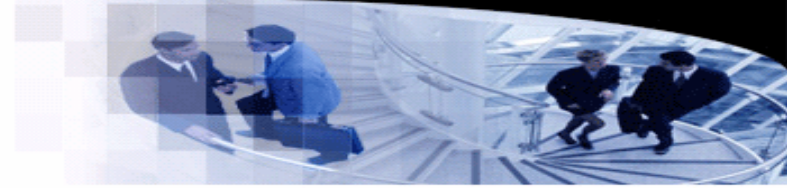
Semantics



- Evaluation

- **Theorem** [Determinacy of one-step evaluation]: If $t \rightarrow t'$ and $t \rightarrow t''$, then $t' = t''$.
- Proof: By induction on a derivation of $t \rightarrow t'$.
 - If the last rule applied in $t \rightarrow t'$ is E-IFTRUE, then t has the form `if t1 then t2 else t3`, where $t1 = \text{true}$.
 - The last rule applied in $t \rightarrow t''$ cannot be E-IFFALSE, as $t1$ does not match `false`.
 - The last rule cannot be E-If either, as the premise requires $t1 \rightarrow t1'$, but `true` does not evaluate to anything according to all evaluation rules.
 - If the last rule is E-IFFALSE, the induction step can be proved similarly.

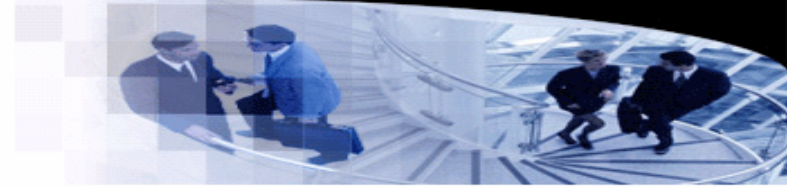
Semantics



- **Evaluation**

- **Theorem** [Determinacy of one-step evaluation]: If $t \rightarrow t'$ and $t \rightarrow t''$, then $t' = t''$.
- **Proof:** By induction on a derivation of $t \rightarrow t'$.
 - If the last rule applied in $t \rightarrow t'$ is E-IF, then t has the form `if t1 then t2 else t3`, where $t1 \rightarrow t1'$.
 - The last rule applied in $t \rightarrow t''$ cannot be E-IfFalse or E-IfTrue.
 - Based on the induction hypothesis, the one-step evaluation of $t1$ has only one result. We know these two last rules has the same premise.
 - Therefore, these two rules draw the same conclusion.
 - ■

Semantics



- **Evaluation**

- A term t is in **normal form** if no evaluation rule applies to it.
- **Theorem**: Every value is in normal form.
- **Theorem**: If t is in normal form, then t is a value.
- The **multi-step evaluation** relation $\rightarrow^*$ is the reflexive, transitive closure of one-step evaluation.
- **Theorem** [Uniqueness of normal forms]: If $t \rightarrow^* u$ and $t \rightarrow^* u'$, where u and u' are both normal forms, then $u = u'$.
- **Theorem** [Termination of Evaluation]: For every term t there is some normal form t' such that $t \rightarrow t'$.

Semantics

- Evaluation

- Dealing with arithmetic expressions.

$\mathbb{B}$ $\mathbb{N}$ (untyped)

Extends $\mathbf{B}$ (3-1)

New syntactic forms

$t ::= \dots$

0

succ t

pred t

iszero t

terms:
constant zero
successor
predecessor
zero test

$v ::= \dots$

nv

values:
numeric value

nv ::=

0

succ nv

numeric values:
zero value
successor value

New evaluation rules

$t \rightarrow t'$

$$\frac{t_1 \rightarrow t'_1}{\text{succ } t_1 \rightarrow \text{succ } t'_1}$$

(E-SUCC)

$$\text{pred } 0 \rightarrow 0$$

(E-PREDZERO)

$$\text{pred } (\text{succ } nv_1) \rightarrow nv_1$$

(E-PREDSUCC)

$$\frac{t_1 \rightarrow t'_1}{\text{pred } t_1 \rightarrow \text{pred } t'_1}$$

(E-PRED)

$$\text{iszero } 0 \rightarrow \text{true}$$

(E-ISZEROZERO)

$$\text{iszero } (\text{succ } nv_1) \rightarrow \text{false}$$

(E-ISZEROSUCC)

$$\frac{t_1 \rightarrow t'_1}{\text{iszero } t_1 \rightarrow \text{iszero } t'_1}$$

(E-ISZERO)

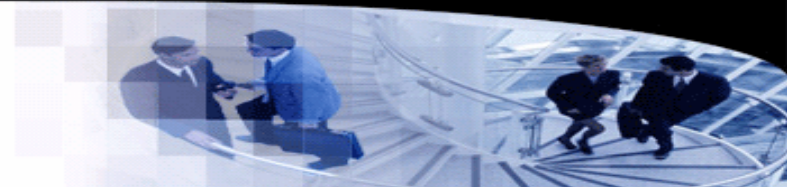
Semantics

- Evaluation

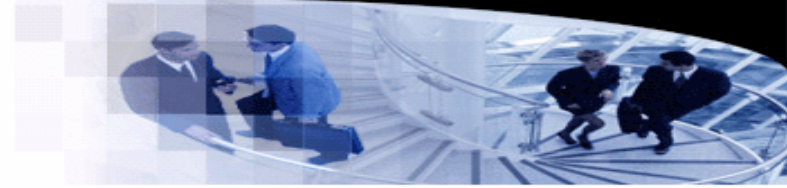
- Example:

- Evaluate $\text{pred} (\text{succ} (\text{pred} 0))$ in one step.

$$\frac{\frac{\frac{}{\text{pred } 0 \rightarrow 0} \text{E-PREDZERO}}{\text{succ} (\text{pred } 0) \rightarrow \text{succ } 0} \text{E-SUCC}}{\text{pred} (\text{succ} (\text{pred } 0)) \rightarrow \text{pred} (\text{succ } 0)} \text{E-PRED}$$



Semantics



- Evaluation

- How to evaluate `succ false`?
- Stuckness: A term is stuck if it is in normal form but not a value.
- Stuckness represents a runtime error for this language.
- It is interpreted as a meaningless state in a machine by operational semantics.
- In practical programming languages, it may correspond to segment faults, execution of illegal instructions, etc.

Implementations

- An implementation in OCaml
 - <https://www.cis.upenn.edu/~bcpierce/tapl/checkers/arith>

Directory Index

Name

 [Makefile](#)
 [core.ml](#)
 [core.mli](#)
 [lexer.mli](#)
 [main.ml](#)
 [parser.mly](#)
 [support.ml](#)
 [support.mli](#)
 [syntax.ml](#)
 [syntax.mli](#)
 [test.f](#)

```
open Format
open Support.Error
open Support.Pervasive
```

```
(* -----
(* Datatypes *)
```

```
type term =
  TmTrue of info
| TmFalse of info
| TmIf of info * term * term * term
| TmZero of info
| TmSucc of info * term
| TmPred of info * term
| TmIsZero of info * term
```

```
type command =
  Import of string
| Eval of info * term
```

```
(* -----
(* Extracting file info *)
```

```
let tmInfo t = match t with
  TmTrue(fi) -> fi
| TmFalse(fi) -> fi
| TmIf(fi,_,_,_) -> fi
| TmZero(fi) -> fi
| TmSucc(fi,_) -> fi
| TmPred(fi,_) -> fi
| TmIsZero(fi,_) -> fi
```

```
let rec eval1 t = match t with
  TmIf(_,TmTrue(_),t2,t3) ->
    t2
| TmIf(_,TmFalse(_),t2,t3) ->
    t3
| TmIf(fi,t1,t2,t3) ->
    let t1' = eval1 t1 in
    TmIf(fi, t1', t2, t3)
| TmSucc(fi,t1) ->
    let t1' = eval1 t1 in
    TmSucc(fi, t1')
| TmPred(_,TmZero(_)) ->
    TmZero(dummyinfo)
| TmPred(_,TmSucc(_,nv1)) when (isnumericval nv1) ->
    nv1
| TmPred(fi,t1) ->
    let t1' = eval1 t1 in
    TmPred(fi, t1')
| TmIsZero(_,TmZero(_)) ->
    TmTrue(dummyinfo)
| TmIsZero(_,TmSucc(_,nv1)) when (isnumericval nv1) ->
    TmFalse(dummyinfo)
| TmIsZero(fi,t1) ->
    let t1' = eval1 t1 in
    TmIsZero(fi, t1')
| _ ->
    raise NoRuleApplies

let rec eval t =
  try let t' = eval1 t
    in eval t'
  with NoRuleApplies -> t
```